Manual de programación

OpenRIN

Documentación técnica para el programador

Prácticas Profesionalizantes III

Autor: Muñiz Cortés, Carlos Santiago - Legajo B00000955-VCO

Versión 1.0

Tabla de contenido

- 1. Introducción
  - 1.1. Tecnologías
- 2. Arquitectura
  - 2.1. Estructura de la solución
  - 2.2. Bases de datos
  - 2.3. Configuración y arranque
- 3. Patrones de diseño
- 4. Excepciones, encapsulación y fábrica
  - 4.1. Excepciones de negocio
  - 4.2. Encapsulación y SqlHelper
  - 4.3. Fábrica de repositorios
- 5. Hilos y concurrencia
  - 5.1. Refresco del panel de monitoreo
  - 5.2. Respaldo programado
  - 5.3. Estado compartido protegido con lock
- 6. Seguridad
- 7. Idiomas
  - 7.1. Agregar un texto
  - 7.2. Agregar un idioma
- 8. Compilar, ejecutar y probar
  - 8.1. Compilar
  - 8.2. Pruebas automáticas
  - 8.3. Generar el instalador
  - 8.4. Configuración
- 9. Cómo agregar una función
  - 9.1. Agregar un módulo
  - 9.2. Agregar una entidad
  - 9.3. Cambiar una regla de negocio
- 10. Referencia de clases
  - 10.1. Espacio de nombres Negocio.BLL
  - 10.2. Espacio de nombres Negocio.DAL.Auditoria
  - 10.3. Espacio de nombres Negocio.DAL.Context
  - 10.4. Espacio de nombres Negocio.DAL.Repositorios

- 10.5. Espacio de nombres Negocio.DomainModel
- 10.6. Espacio de nombres Negocio.DomainModel.Enums
- 10.7. Espacio de nombres Negocio.DomainModel.Exceptions
- 10.8. Espacio de nombres Negocio.UI
- 10.9. Espacio de nombres Services.BLL
- 10.10. Espacio de nombres Services.BLL.Backups
- 10.11. Espacio de nombres Services.BLL.ExceptionManagement
- 10.12. Espacio de nombres Services.BLL.Infrastructure
- 10.13. Espacio de nombres Services.BLL.Localization
- 10.14. Espacio de nombres Services.DAL.Factory
- 10.15. Espacio de nombres Services.DAL.Implementations
- 10.16. Espacio de nombres Services.DAL.Interfaces
- 10.17. Espacio de nombres Services.DAL.Tools
- 10.18. Espacio de nombres Services.DomainModel
- 10.19. Espacio de nombres Services.DomainModel.Exceptions
- 10.20. Espacio de nombres Services.DomainModel.Validacion
- 10.21. Espacio de nombres Services.Facade

1. Introducción

Este manual está dirigido a quien tenga que mantener o ampliar OpenRIN. Explica cómo está organizado el código, qué patrones usa, cómo se manejan los hilos, la seguridad y los idiomas, cómo compilar y probar, y cómo agregar una función. Al final tiene la referencia de clases, generada a partir de los comentarios de documentación XML del propio código.

Para el uso del programa ver el Manual de usuario; para instalarlo, el Manual de instalación.

1.1. Tecnologías

Componente	Tecnología
Lenguaje y plataforma	C# 12 sobre .NET 8 (net8.0-windows)
Interfaz	Windows Forms, con controles propios por módulo
Acceso a datos del negocio	Entity Framework Core 8 con repositorios y un único punto de guardado
Acceso a datos de servicios	ADO.NET (Microsoft.Data.SqlClient) con comandos parametrizados
Base de datos	SQL Server 2019 o superior (probado con SQL Server 2022 Express)
PDF	QuestPDF (licencia comunitaria)
Criptografía	System.Security.Cryptography (AES, PBKDF2) y DPAPI (ProtectedData)
Instalador	PowerShell + IExpress, con el runtime de .NET incluido

2. Arquitectura

El sistema está dividido en capas. Cada capa solo conoce a las que están debajo; la interfaz nunca toca la base de datos.

2.1. Estructura de la solución

Proyecto	Capa	Qué contiene
Negocio.DomainModel	Dominio	Entidades (Paciente, HistoriaClinica, MedicionRIN, Alerta, Turno...), enumeraciones, excepciones y registros auxiliares.
Negocio.DAL	Acceso a datos	NegocioDbContext de EF Core, repositorios, auditoría de cambios y migraciones.
Negocio.BLL	Lógica de negocio	Clases *Logic con las reglas de cada módulo: criticidad, tendencia, agenda, seguimiento, reportes e integridad.
Services	Transversal	Seguridad, criptografía, localización, bitácora, respaldos, transacciones y manejo de excepciones, con su fachada.
Negocio.UI	Interfaz	Formularios y un UserControl por módulo.
Herramientas	Utilitario	Comandos sembrar, respaldo e integridad.
Pruebas	Pruebas	Programa de consola con 103 comprobaciones contra una base real.

Las dependencias van así: Negocio.UI → Negocio.BLL → Negocio.DAL → Negocio.DomainModel, y Services lo usan todas las capas a través de sus fachadas.

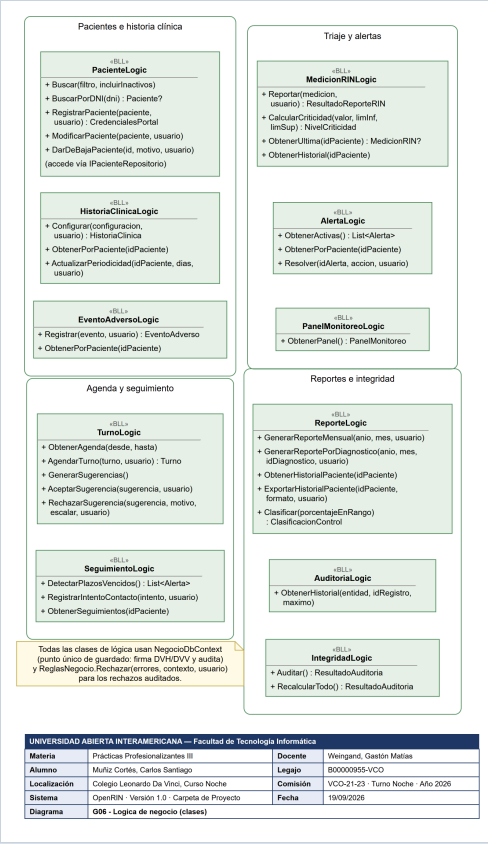


Diagrama de clases de la capa de lógica de negocio (draw.io).

2.2. Bases de datos

- OpenRIN_Negocio: 13 tablas (Pacientes, ObrasSociales, Diagnosticos, HistoriasClinicas, MedicionesRIN, Alertas, Turnos, Seguimientos, EventosAdversos, Usuarios, ReportesEstadisticos, DigitosVerificadores y AuditoriaCambios). El esquema lo administra Entity Framework mediante migraciones.

- OpenRIN_Services: Usuarios, Permisos, PerfilesPermisos, Idiomas, Textos, Logs y Respaldos, accedidas con ADO.NET a través de una única clase ayudante (SqlHelper).

Los scripts equivalentes están en la carpeta «4 - Base de Datos». El de negocio se regenera con:

```
dotnet ef migrations script --idempotent --project Negocio.DAL --startup-project Negocio.DAL
-o Negocio.DAL/Tools/script.sql
```

Once tablas llevan la columna DVH (todas menos DigitosVerificadores y AuditoriaCambios). Las relaciones se declaran con 16 claves foráneas en OpenRIN_Negocio (todas las columnas que referencian pacientes, historias, mediciones o usuarios, con borrado restringido) y 2 en OpenRIN_Services (Permisos hacia su grupo padre y Textos hacia Idiomas). PerfilesPermisos y la preferencia de idioma del usuario son referencias lógicas, sin clave foránea. Los diagramas entidad-relación (hojas A, B y C) se generan leyendo el esquema real con Diagramas/_fuente/der.mjs y están en la carpeta «3 - Modelo de diagramas» y en la Carpeta de Proyecto.

2.3. Configuración y arranque

Program.Main registra los manejadores globales de excepciones, establece el idioma base, verifica la integridad de la base (VerificacionArranque.Auditar) y, si está todo bien, muestra LoginForm y luego PrincipalForm. Si la verificación falla se muestra IntegridadVulneradaForm y no se permite el acceso.

Las cadenas de conexión se leen del archivo .dll.config mediante ConfigurationManager (claves ServicesDB, NegocioDB y BackupString). Las clases que implementan cada repositorio también se indican allí (claves «Repositorio.NombreDeLaInterfaz») y la fábrica las crea por reflexión.

3. Patrones de diseño

Patrón	Dónde	Para qué
Singleton	SesionActual	Una única sesión de usuario, creada de forma perezosa y protegida con lock.
Composite	Permiso, PermisoAtomico, PermisoCompuesto	Árbol de permisos: las hojas y los grupos responden igual a Incluye().
Observer	LocalizationLogic.IdiomaCambiado	Las vistas se suscriben y refrescan sus textos al cambiar el idioma.
Repository	IRepositorioGenerico<T>, IPacienteRepositorio	Aislar el acceso a datos de la lógica de negocio.
Facade	Services/Facade/*	Punto de entrada estático a los servicios técnicos.
Factory (con reflexión)	RepositorioFactory	Crea cada repositorio a partir del nombre de clase de la configuración; valida que exista, sea concreta, implemente la interfaz y tenga el constructor pedido.
Unit of Work (punto único de guardado)	NegocioDbContext.SaveChanges	Firmar DVH/DVV y auditar en una sola operación.

4. Excepciones, encapsulación y fábrica

4.1. Excepciones de negocio

Todas las excepciones controladas derivan de ReglaNegocioException (Services.DomainModel.Exceptions). La pantalla muestra su mensaje tal cual y no el texto genérico «Ocurrió un error inesperado». Las derivadas son: DatoInvalidoException, RegistroNoEncontradoException, RegistroDuplicadoException, AccesoADatosException, RespaldoException, ConfiguracionInvalidaException, IntegridadVulneradaException y ValidacionNegocioException (reglas del negocio clínico).

No se lanzan ArgumentException ni InvalidOperationException genéricas: cada fallo usa la excepción que lo describe.

4.2. Encapsulación y SqlHelper

Las clases de acceso a datos son internal: SqlBackupRepository, SqlUsuarioRepository, SqlLoggerRepository, SqlRespaldoRegistroRepository, PacienteRepositorio y las demás. Quedan públicos solo la fábrica, las interfaces y el dominio. Los proyectos de pruebas ven lo interno mediante InternalsVisibleTo.

SqlHelper es la única clase que abre conexiones, arma comandos parametrizados y maneja transacciones en OpenRIN_Services; convierte los errores de SQL Server en AccesoADatosException. Todos los repositorios SQL la usan.

4.3. Fábrica de repositorios

RepositorioFactory.Crear<IInterfaz>() lee la clave «Repositorio.IInterfaz» del archivo de configuración, busca la clase por su nombre completo y la crea por reflexión. Si la clave falta, la clase no existe, es abstracta, no implementa la interfaz o no tiene el constructor pedido, lanza ConfiguracionInvalidaException con el detalle. Para repositorios con parámetros (por ejemplo, IPacienteRepositorio recibe el NegocioDbContext) se pasan como argumentos.

```
<add key="Repositorio.IBackupRepository"
value="Services.DAL.Implementations.SqlBackupRepository" />
```

5. Hilos y concurrencia

OpenRIN es una aplicación de un solo puesto y casi todo corre en el hilo de la interfaz. Hay tres puntos con concurrencia que conviene conocer.

5.1. Refresco del panel de monitoreo

PanelTriajeControl usa un System.Windows.Forms.Timer de 30 segundos. Ese temporizador dispara en el hilo de la interfaz, por lo que no hace falta sincronizar el acceso a los controles. Se pausa al cambiar de módulo (Pausar) y se reanuda al volver (Reanudar).

5.2. Respaldo programado

BackupLogic.ProgramarRespaldoDiario crea un System.Threading.Timer que dispara en un hilo del grupo de subprocesos. El acceso al temporizador está protegido con lock (Candado). En producción el respaldo no depende de la aplicación abierta: lo ejecuta la tarea programada de Windows con OpenRIN.Herramientas.exe respaldo-auto, que solo respalda si los datos están sanos (una vez por día, conservando los 10 últimos).

5.3. Estado compartido protegido con lock

- SesionActual: instancia única con doble verificación y lock.
- LocalizationLogic: diccionario del idioma activo, lista de idiomas y suscriptores, protegidos con lock (Candado).
- BitacoraLogic y TransaccionDAL: escritura de la bitácora y transacción activa, protegidas con lock.

> **Nota:** Si agrega trabajo en segundo plano, no toque controles desde otro hilo: use Control.Invoke para volver al hilo de la interfaz.

6. Seguridad

- Contraseñas y respuestas de seguridad: PBKDF2-SHA256, 120 000 iteraciones y sal aleatoria de 16 bytes; formato «sal.iteraciones.hash»; comparación en tiempo constante.
- Cifrado simétrico: AES-CBC de 256 bits (CryptographyService). La clave se genera la primera vez y se guarda con DPAPI en %LOCALAPPDATA%\OpenRIN\enc.key.
- Anti fuerza bruta: 5 intentos fallidos bloquean la cuenta 15 minutos.
- Respaldos: BACKUP con COPY_ONLY y CHECKSUM, verificado con RESTORE VERIFYONLY; tabla Respaldos con origen, estado sano o no, tamaño y usuario; si SQL Server no puede escribir en la carpeta elegida, respalda en la suya y la aplicación copia el archivo. Restaurar exige verificar el archivo, hace antes una copia previa de seguridad y usa SINGLE_USER con RESTORE ... WITH REPLACE.
- Recuperación por dígitos verificadores: si el arranque detecta alteraciones se registra «Integridad — Alteración detectada» en la bitácora, se bloquea el ingreso y IntegridadVulneradaForm lista los registros alterados; un usuario con el permiso SI004 inicia sesión y restaura un respaldo sano, restaura otro .bak o recalcula los dígitos con confirmación. Todo queda en la bitácora.
- Acceso a base: autenticación integrada de Windows; las consultas ADO.NET son siempre parametrizadas.
- Permisos: cada módulo y acción se habilita consultando el árbol de permisos por código (PA001, PO001, SI001...).
- Integridad: dígitos verificadores horizontales (por fila) y verticales (por tabla). SaveChanges trabaja en tres fases: persistir, firmar el DVH de cada fila y recalcular el DVV de cada tabla tocada.
- Control de cambios: AuditoriaDeCambios serializa en JSON el estado anterior y el nuevo; AuditoriaLogic.Restaurar recompone un estado anterior y lo vuelve a firmar.

> **Nota:** La clave AES existe como servicio, pero hoy no se aplica a campos de la base: las contraseñas y respuestas de seguridad se almacenan con hash PBKDF2.

7. Idiomas

Los textos de los idiomas entregados están en archivos .resx: Textos.resx (idioma base, español) y uno por idioma (Textos.en.resx, Textos.zh-CN.resx). Se compilan en ensamblados satélite (Services.resources.dll), una carpeta por cultura junto al programa, y el instalador incluye esas carpetas. La base (tablas Idiomas y Textos) solo agrega idiomas nuevos y leyendas corregidas, que tienen prioridad sobre las del .resx. LocalizationService.ObtenerTexto(clave) devuelve el texto del idioma activo, o la clave si no existe. Se entregan es, en y zh-CN. EstablecerIdioma(codigo) cambia el idioma en caliente y notifica a los suscriptores.

7.1. Agregar un texto

1. Elija una clave jerárquica, por ejemplo agenda.nuevaColumna.
2. Agregue la clave en los tres archivos: Services/BLL/Localization/Textos.resx, Textos.en.resx y Textos.zh-CN.resx, y vuelva a compilar.
3. Use Texto("agenda.nuevaColumna") en el formulario y vuelva a aplicar los textos en AplicarTextos().

7.2. Agregar un idioma

Desde Sistema y Bitácora > Idiomas, sin recompilar (idioma nuevo con sus leyendas en la base). Para un idioma entregado con el programa, agregue Textos.<cultura>.resx con todas las claves y registre el idioma en la tabla Idiomas.

8. Compilar, ejecutar y probar

8.1. Compilar

1. Instale el SDK de .NET 8 o superior y SQL Server (Express alcanza).
2. Copie el proyecto en una ruta corta (por ejemplo C:\OpenRIN): las rutas largas rompen la compilación.
3. Cree las bases y cargue los scripts (Manual de instalación, sección 6).
4. Cree la configuración junto al ejecutable (apartado 7.4).

```
cd Codigo
dotnet build PlataformaACO.sln
dotnet run --project Negocio.UI
```

8.2. Pruebas automáticas

El proyecto Pruebas recorre los casos de uso contra una base real y verifica 103 comprobaciones: idiomas, seguridad, pacientes, historia clínica, RIN y criticidad, alertas, eventos, agenda, seguimiento, reportes, exportación, control de cambios, integridad, respaldo (COPY_ONLY, CHECKSUM, registro, retención y restauración), recuperación con dígitos verificadores, excepciones, fábrica de repositorios e idiomas con .resx. Termina con código 0 si todas pasan.

```
copy Pruebas\App.config.ejemplo Pruebas\App.config
dotnet run --project Pruebas
```

8.3. Generar el instalador

Instalador/Construir_Setup.ps1 publica Negocio.UI y Herramientas como autocontenidos (win-x64), comprime la aplicación, copia los scripts SQL y la ayuda en línea y genera OpenRIN-Setup.exe con IExpress.

```
powershell -ExecutionPolicy Bypass -File Instalador\Construir_Setup.ps1
```

8.4. Configuración

Cree Negocio.UI/App.config tomando como modelo App.config.example. En .NET 8 se copia como Negocio.UI.dll.config. Ejemplo con SQL Server local y autenticación de Windows:

```
<connectionStrings>
  <add name="ServicesDB"
connectionString="Server=.\SQLEXPRESS;Database=OpenRIN_Services;Integrated
Security=True;TrustServerCertificate=True;" />
  <add name="NegocioDB"
connectionString="Server=.\SQLEXPRESS;Database=OpenRIN_Negocio;Integrated
Security=True;TrustServerCertificate=True;" />
  <add name="BackupString" connectionString="Server=.\SQLEXPRESS;Database=master;Integrated
Security=True;TrustServerCertificate=True;" />
</connectionStrings>
```

9. Cómo agregar una función

9.1. Agregar un módulo

1. Cree la clase de lógica en Negocio.BLL, que reciba el NegocioDbContext.
2. Cree el UserControl en Negocio.UI.
3. En PrincipalForm.ConstruirMenu agregue AgregarModulo("mod.nuevo", sysadmin || tiene("PA0xx")) y la rama correspondiente en MostrarModulo.
4. Agregue el permiso en permisos.sql, su texto en Textos.resx (y en Textos.en.resx y Textos.zh-CN.resx) y el tema de ayuda en Documentacion/_fuente.

9.2. Agregar una entidad

1. Cree la clase en Negocio.DomainModel/Entities.
2. Agregue el DbSet y su configuración a NegocioDbContext.
3. Si debe firmarse, agréguela a IntegridadLogic.
4. Genere la migración y regenere el script de esquema.

```
dotnet ef migrations add NombreMigracion --project Negocio.DAL --startup-project Negocio.DAL
```

9.3. Cambiar una regla de negocio

Las constantes están en la lógica correspondiente: MedicionRINLogic (DistanciaAlta = 0,5; MargenBorde = 0,1), SeguimientoLogic (UmbralIntentosFallidos = 3) y ReporteLogic (UmbralOptimo = 80; UmbralSuboptimo = 50). Actualice también las pruebas y los diagramas.

10. Referencia de clases

Generada a partir de los comentarios de documentación XML del código, agrupada por espacio de nombres.

Espacio de nombres Negocio.BLL

Tipo	Nombre	Descripción
class	AlertaLogic	Lógica de alertas (REQ-FUNC-007): las alertas permanecen activas — destacadas en el panel— hasta que el médico registra una acción de resolución. Incluye consultas de alertas activas y por paciente, y el cierre auditado de la alerta.

Tipo	Nombre	Descripción
class	AuditoriaLogic	Control de cambios (T06b): expone el historial de auditoría por entidad y registro, y permite recomponer el estado anterior de un objeto a partir de la fotografía previa. La restauración pasa por el mismo punto de guardado que el resto del sistema: la fila vuelve a firmarse (DVH/DVV) y la propia restauración queda auditada como un cambio más.
record	CredencialesPortal	Credenciales generadas para el portal de pacientes al dar el alta.
class	EventoAdversoLogic	Lógica de registro de eventos adversos (REQ-FUNC-003): verifica que el paciente esté activo y posea historia clínica configurada, valida que la fecha no sea futura y asocia el evento al historial clínico del paciente.
record	ExportacionHistorial	Resultado de una exportación de historial: nombre de archivo sugerido y registros a renderizar.
class	HistoriaClinicaLogic	Lógica de configuración de la historia clínica hematológica (REQ-FUNC-002): valida el rango terapéutico (inferior < superior), la medicación, la periodicidad y las fechas de control, y mantiene una única HC por paciente. Es el insumo indispensable del cálculo de criticidad de los reportes de RIN.
class	IntegridadLogic	Auditoría y reparación de integridad (DVH/DVV): verifica que el dígito horizontal de cada fila corresponda a sus datos y que el dígito vertical de cada tabla coincida con el conjunto de sus filas. La reparación (recalculación total) queda disponible para restablecer las firmas tras una detección.
class	MedicionRINLogic	Lógica de triaje (REQ-FUNC-004/005/008): ingesta de valores de RIN con validaciones (rango biológicamente posible, fecha no futura, paciente activo con HC configurada), cálculo automático de criticidad (Baja dentro del rango · Media fuera por menos de ±0.5 · Alta fuera por ±0.5 o más), detección de tendencia peligrosa (dos reportes consecutivos en el borde ±0.1) y disparo automático de la alerta urgente.
class	PacienteLogic	Lógica del ciclo de vida de pacientes (REQ-FUNC-001): alta con validaciones y credenciales únicas del portal (usuario = DNI), modificación con control de duplicidad y baja lógica con deshabilitación de credenciales sin borrar el historial clínico.
class	PanelMonitoreoLogic	Panel de monitoreo en tiempo real (REQ-FUNC-006): ranking de pacientes activos ordenado por criticidad (Alta → Media → Baja) y, dentro de cada nivel, por antigüedad del último reporte (los más antiguos primero: llevan más tiempo sin reevaluación). Los pacientes sin reportes quedan al final. El panel se recalcula en cada consulta, de modo que la interfaz siempre muestra el estado vigente.
record	RegistroAlterado	Un registro (o una tabla completa, con Id 0) cuyos dígitos verificadores no coinciden.
class	ReglasNegocio	Reglas transversales de la capa de lógica de negocio: rechazo auditado de operaciones inválidas. Toda validación fallida queda registrada en bitácora con nivel Warning (capa Negocio), tal como exige el REQ-ARQ-007.
class	ReporteLogic	Lógica de reportes estadísticos (REQ-FUNC-011/012/013): reporte mensual de eficacia (% de mediciones dentro del rango terapéutico por paciente, clasificación óptimo / subóptimo / deficiente y consolidado global almacenado), reporte agrupado por diagnóstico, historial clínico unificado y exportación auditada.

Tipo	Nombre	Descripción
record	ResultadoAuditoria	Resultado de una auditoría de integridad (DVH/DVV).
record	ResultadoIntentoContacto	Resultado de registrar un intento de contacto del protocolo de seguimiento.
record	ResultadoReasignacion	Resultado de aceptar una reasignación: el turno nuevo creado y las notificaciones emitidas a los pacientes.
record	ResultadoReporteRIN	Resultado del reporte de una medición: la medición clasificada y, si correspondió, la alerta generada.
class	SeguimientoLogic	Lógica de seguimiento clínico (REQ-FUNC-010): detecta plazos de reporte vencidos según la periodicidad configurada en la HC (último RIN + periodicidad), genera alertas de ausencia idempotentes, administra el protocolo progresivo de contacto (umbral de intentos fallidos → escalado al médico) y registra las decisiones clínicas, que pueden actualizar automáticamente la periodicidad de la HC.
class	TurnoLogic	Lógica de agenda y turnos (REQ-FUNC-009): agendamiento con validaciones (paciente activo, médico habilitado, fecha futura, sin doble reserva), y reasignación inteligente que cruza pacientes ESTABLES con turno (última criticidad Baja) contra pacientes CRÍTICOS sin turno (última criticidad Alta). La aceptación libera el turno del estable y lo asigna al crítico notificando a ambos; el rechazo con duda clínica se escala al médico.

Espacio de nombres Negocio.DAL.Auditoria

Tipo	Nombre	Descripción
class	AuditoriaDeCambios	Utilidades de serialización para el control de cambios (T06b): convierte el estado de una fila (valores de EF) a JSON fiel al tipo original, y aplica un JSON de vuelta sobre una entidad para recomponer su estado anterior. Los enums se guardan por nombre y los decimales/fechas conservan su tipo para una restauración exacta.

Espacio de nombres Negocio.DAL.Context

Tipo	Nombre	Descripción
class	DesignTimeFactory	Fábrica de diseño para las herramientas de Entity Framework Core (dotnet ef): permite crear y aplicar migraciones usando autenticación integrada de Windows contra el servidor local. La aplicación en runtime usa la cadena "NegocioDB" de su propia configuración (con el login de mínimo privilegio usr_negocio).
class	NegocioDbContext	Contexto de Entity Framework Core del módulo Negocio: mapea las entidades del dominio sobre la base OpenRIN_Negocio, con índices de unicidad (incluido el filtrado de DNI entre pacientes activos), conversiones de enums a texto y datos semilla de catálogos. Integra la firma de integridad (DVH por fila / DVV por tabla) en un único punto: cada SaveChanges firma automáticamente las filas escritas y recalcula los dígitos verticales de las tablas afectadas.

Espacio de nombres Negocio.DAL.Repositorios

Tipo	Nombre	Descripción
interface	IPacienteRepositorio	Operaciones específicas de acceso a datos de pacientes (REQ-FUNC-001).

Tipo	Nombre	Descripción
interface	IRepositorioGenerico	Contrato genérico de repositorio de la capa de datos del módulo Negocio (EF Core).
class	PacienteRepositorio	Repositorio de pacientes sobre EF Core (REQ-FUNC-001): altas, búsquedas por documento y verificación de unicidad entre pacientes activos.
class	RepositorioGenerico	Implementación genérica de repositorio sobre EF Core: cada operación de escritura persiste de inmediato (SaveChanges). Los repositorios específicos heredan de esta clase y agregan consultas propias del dominio.

Espacio de nombres Negocio.DomainModel

Tipo	Nombre	Descripción
class	Alerta	Notificación generada ante valores críticos o ausencias de reporte (REQ-FUNC-007/008/010). Permanece Activa hasta que el médico registra una acción de resolución; si el protocolo de contacto se agota sin éxito, la alerta se escala al médico.
class	CambioAuditado	Registro de auditoría de cambios (requisito T06b): documenta quién, cuándo y qué cambió sobre cada entidad del negocio, guardando el estado anterior y el nuevo en formato JSON. Esta tabla no se firma con DVH/DVV a propósito: es ella misma la evidencia de auditoría y su alteración se detecta por su propio historial, no por dígitos verificadores.
class	Diagnostico	Catálogo de patologías o indicaciones de anticoagulación usadas en las historias clínicas (REQ-FUNC-002 / REQ-FUNC-012).
class	DigitosVerificadores	Dígito verificador vertical (DVV) por tabla (integridad de datos): firma SHA-256 del conjunto de DVH de todas las filas de una tabla, recalculada en cada escritura.
class	DigitoVerificador	Cálculo de dígitos verificadores (integridad de datos, DVH/DVV): el DVH firma los campos escalares de una fila y el DVV firma el conjunto de DVH de una tabla. La MISMA rutina se usa al escribir (contexto EF) y al auditar (BLL), garantizando firmas reproducibles.
class	EventoAdverso	Incidente clínico relevante registrado en la historia clínica del paciente (REQ-FUNC-003): hemorragias, trombosis, reacciones. La fecha no puede ser futura.
class	FilaPanelMonitoreo	Fila del panel de monitoreo (REQ-FUNC-006): datos de un paciente activo con su último valor de RIN, la criticidad vigente y el estado de alerta para el ranking visual.
class	FilaReporteDiagnostico	Fila del reporte de eficacia por diagnóstico (REQ-FUNC-012): agrupa a los pacientes según la patología registrada en su HC y resume su desempeño en el período.
class	HistoriaClinica	Configuración hematológica del paciente (REQ-FUNC-002): rango terapéutico objetivo de RIN, medicación y periodicidad esperada de reporte. Es el insumo indispensable para el cálculo de criticidad de los reportes de RIN.
class	ItemHistorial	Registro del historial clínico unificado de un paciente (REQ-FUNC-013): una medición de RIN, una alerta o un evento adverso, presentados cronológicamente.
class	MedicionRIN	Registro individual de un valor de RIN reportado (REQ-FUNC-004). Almacena la fecha/hora, el canal y el usuario que realizó la carga, y queda disponible para el módulo de cálculo de criticidad.
class	ObraSocial	Entidad aseguradora que cubre al paciente (obra social / prepaga).

Tipo	Nombre	Descripción
class	Paciente	Persona anticoagulada registrada en el centro (REQ-FUNC-001). La baja es lógica: el estado pasa a Inactivo y las credenciales se deshabilitan, pero el historial clínico se conserva.
class	ReporteEstadistico	Informe estadístico de eficacia del tratamiento (REQ-FUNC-011): consolidado mensual con indicadores por paciente, clasificación de control y detalle serializado para trazabilidad. Alimenta también el reporte por diagnóstico (REQ-FUNC-012).
class	Seguimiento	Registro de intentos de contacto y decisiones clínicas del seguimiento proactivo (REQ-FUNC-010). Si un intento es exitoso, la alerta de ausencia asociada se cierra; al agotar el umbral de intentos fallidos, el caso se escala al médico, quien puede registrar su decisión clínica (incluyendo actualizar la periodicidad de la HC).
class	SugerenciaReasignacion	Sugerencia de reasignación inteligente de turnos (REQ-FUNC-009): propone mover el turno de un paciente estable a un paciente crítico que no posee turno, para maximizar la eficiencia clínica de la agenda.
class	Turno	Cita presencial programada en la agenda médica (REQ-FUNC-009). Un turno puede entrar en estado EnReasignacion como candidato dentro de una sugerencia de reasignación inteligente entre pacientes estables y críticos.
class	Usuario	Persona autenticada con un perfil específico (base del módulo de autenticación, REQ-ARQ-006). El registro técnico de credenciales (hash, intentos fallidos, idioma) vive en OpenRIN_Services; esta entidad representa la identidad de negocio, enlazada por NombreUsuario.

Espacio de nombres Negocio.DomainModel.Enums

Tipo	Nombre	Descripción
enum	CanalReporteRIN	Canal por el que se recibió un reporte de RIN (REQ-FUNC-004): carga directa en la plataforma o carga manual tras un reporte telefónico.
enum	ClasificacionControl	Clasificación del control de un paciente según su porcentaje de mediciones dentro del rango terapéutico (REQ-FUNC-011): óptimo, subóptimo o deficiente.
enum	DecisionClinica	Decisión clínica del médico tras agotar el protocolo de contacto (REQ-FUNC-010).
enum	EstadoAlerta	Estado de una alerta (REQ-FUNC-007): permanece Activa hasta que el médico registra una acción de resolución.
enum	EstadoPaciente	Estado del ciclo de vida de un paciente (REQ-FUNC-001). La baja es lógica: el historial clínico nunca se elimina.
enum	EstadoTurno	Estado de un turno presencial en la agenda médica (REQ-FUNC-009).
enum	GravedadEvento	Gravedad de un evento adverso (REQ-FUNC-003).
enum	NivelCriticidad	Nivel de criticidad de una medición de RIN (REQ-FUNC-005): Baja = dentro del rango terapéutico; Media = fuera por menos de ±0.5; Alta = fuera por ±0.5 o más (o por tendencia peligrosa, REQ-FUNC-008).
enum	ResultadoContacto	Resultado de un intento de contacto (REQ-FUNC-010): si es Exitoso, la alerta de ausencia asociada se cierra.
enum	TipoAlerta	Motivo de generación de una alerta (REQ-FUNC-007/008/010).

Tipo	Nombre	Descripción
enum	TipoContacto	Medio utilizado en un intento de contacto del protocolo de seguimiento (REQ-FUNC-010).
enum	TipoEventoAdverso	Tipo de incidente clínico relevante registrado en la historia clínica (REQ-FUNC-003).

Espacio de nombres Negocio.DomainModel.Exceptions

Tipo	Nombre	Descripción
class	ValidacionNegocioException	Error de validación del negocio: agrupa los mensajes que explican por qué una operación fue rechazada (para mostrar al usuario y auditar en bitácora).

Espacio de nombres Negocio.UI

Tipo	Nombre	Descripción
class	AgendaControl	Agenda y turnos (REQ-FUNC-009): vista de la agenda médica por período y panel de sugerencias de reasignación inteligente (estables con turno → críticos sin turno), con aceptación (libera + asigna + notifica) o rechazo (escalable al médico).
class	AgendarTurnoForm	Diálogo para agendar un turno presencial (REQ-FUNC-009): selección de paciente activo, médico habilitado, fecha y hora, con las validaciones de agenda de la BLL.
class	CredencialesPortalForm	Muestra una única vez las credenciales del portal generadas al dar de alta un paciente (REQ-FUNC-001): usuario (DNI) y contraseña temporal, con copiado al portapapeles.
class	EventoAdversoForm	Diálogo de alta de un evento adverso (REQ-FUNC-003): fecha (no futura), tipo, gravedad, descripción y acción tomada, con las validaciones de la BLL.
class	EventosAdversosControl	Registro de eventos adversos (REQ-FUNC-003): búsqueda del paciente por documento, consulta de sus eventos registrados (o de los últimos eventos de todos los pacientes) y alta de nuevos incidentes con tipo, gravedad, descripción y acción tomada.
class	Exportadores	Renderizadores de exportación de la capa de presentación (REQ-FUNC-013): generan el archivo real del historial clínico unificado en PDF (con la librería de terceros QuestPDF, requisito A02), en Excel (.xlsx OOXML generado como paquete ZIP) o en JSON serializado (A03: archivo serializado con información relevante).
class	HistoriaClinicaForm	Historia clínica hematológica de un paciente (REQ-FUNC-002): creación/actualización por perfiles con permiso HISTORIA_CLINICA, o vista de solo lectura para el resto.
class	IdiomaNuevoForm	Alta de idioma (T05): diálogo con código y nombre para incorporar un idioma nuevo desde la administración del sistema. La creación efectiva la realiza la fachada de localización, que valida y audita la operación.

Tipo	Nombre	Descripción
class	IntegridadVulneradaForm	Pantalla de bloqueo por integridad (T08): reemplaza a la ventana de log-in cuando la verificación de arranque detecta alteraciones en la base realizadas por fuera del sistema. Muestra la lista de registros alterados y permite recuperar la base: un usuario con permiso de respaldos (SI004) inicia sesión, elige un respaldo sano de la lista (o otro archivo .bak) y lo restaura, o recalcula los dígitos con confirmación. Todo queda en la bitácora (capa «Integridad»). Devuelve si la base quedó sana.
class	LoginForm	Pantalla de inicio de sesión: autentica contra el módulo de seguridad de Services (REQ-ARQ-006), muestra el logo institucional y permite cambiar el idioma en vivo (REQ-ARQ-001) y recuperar la contraseña con pregunta de seguridad.
class	PacienteEditorForm	Alta (REQ-FUNC-001) y modificación de pacientes. En el alta se generan las credenciales únicas del portal y se muestran una única vez para entregarlas al paciente.
class	PacientesControl	Pantalla de gestión de pacientes e historias clínicas (REQ-FUNC-001/002): búsqueda por nombre/documento, alta con credenciales del portal, modificación, baja lógica con motivo y acceso a la historia clínica. Los botones de gestión se habilitan según los permisos del perfil (REQ-ARQ-006).
class	PanelTriajeControl	Panel de Monitoreo / Triaje (REQ-FUNC-006): ranking en tiempo real de pacientes activos ordenado por criticidad y antigüedad del último reporte, con indicadores visuales (colores por criticidad, alerta activa y tendencia peligrosa). Se refresca automáticamente cada 30 segundos y de forma manual con "Actualizar".
class	PortalControl	Portal del paciente: vista para la persona anticoagulada autenticada (REQ-FUNC-005/013): reporta su propio RIN por el canal digital y consulta su historial clínico unificado. El vínculo sesión → paciente se resuelve por Paciente.IdUsuarioPortal.
class	PrincipalForm	Ventana principal (shell): cabecera de sesión con indicador de integridad (DVH/DVV), menú de módulos habilitados según los permisos del perfil (REQ-ARQ-006) y área de contenido donde se incrustan las pantallas de cada módulo (UserControls). Permite cambiar el idioma en vivo (REQ-ARQ-001).
class	RecepcionRINControl	Recepción de RIN (REQ-FUNC-004/005): búsqueda del paciente por documento, carga manual del valor (canal digital o telefónico) y resultado inmediato del triaje: clasificación de criticidad, reclasificación por tendencia peligrosa y alerta urgente generada.
class	RecuperacionForm	Diálogo de recuperación de contraseña (REQ-NF-001): valida la identidad mediante la pregunta de seguridad configurada (hash) y permite establecer una contraseña nueva.
class	Recursos	Recursos visuales embebidos de OpenRIN: el logo (openrin.ico) se carga desde el ensamblado, sin depender de archivos externos en disco.
class	ReportesControl	Reportes estadísticos (REQ-FUNC-011/012/013): reporte mensual de eficacia con semáforo óptimo/subóptimo/deficiente, reporte de eficacia agrupado por diagnóstico (destacando los grupos con bajo desempeño), historial clínico unificado por paciente y exportación auditada del historial a PDF/Excel.

Tipo	Nombre	Descripción
class	RespaldosPanel	Pestaña «Respaldos» de Sistema y Bitácora: elegir la carpeta, respaldar ahora (COPY_ONLY + CHECKSUM, verificado), verificar un archivo, ver la tabla Respaldos (origen, estado sano o no, tamaño y usuario) y restaurar con copia previa de seguridad. Requiere el permiso SI004; la restauración de la base de negocio queda en la bitácora.
class	SeguimientoControl	Seguimiento clínico (REQ-FUNC-010): casos con reporte vencido (alertas de ausencia activas), protocolo progresivo de contacto (umbral de intentos fallidos → escalado al médico) y registro de decisiones clínicas (que pueden actualizar la periodicidad de la HC).
class	SistemaControl	Administración del sistema (REQ-ARQ-004/006, solo perfil sysadmin): gestión de usuarios (listado, alta vía la API de seguridad, habilitación/deshabilitación) y consulta de la bitácora centralizada con filtro por nivel mínimo.
class	TextoDialogoForm	Diálogo genérico para solicitar un texto corto (por ejemplo, el motivo de una baja).
class	UsuarioNuevoForm	Diálogo de alta de usuario del sistema (pantalla Sistema): registra la cuenta a través de la API real de seguridad (hash PBKDF2 + validaciones de contraseña), con pregunta y respuesta de seguridad opcionales para el flujo de recuperación de contraseña.
class	VerificacionArranque	Verificación de integridad (DVH/DVV) ejecutada en el arranque de la aplicación, antes de mostrar cualquier ventana (requisito T08). Si detecta alteraciones, el flujo principal muestra la pantalla de alerta y no habilita el ingreso.

Espacio de nombres Services.BLL

Tipo	Nombre	Descripción
class	BitacoraLogic	Lógica de bitácora (REQ-ARQ-003): filtra por el nivel mínimo configurado (app.config), persiste en la tabla Logs y cae al archivo de respaldo si la base de datos falla. Nunca propaga excepciones hacia el invocante. Expone además la consulta de las últimas entradas para la pantalla de administración (Sistema).
class	ConsistenciaLogic	Lógica de consistencia (REQ-ARQ-007): validación de entidades por atributos con reflexión ([Requerido], [Unico], [Rango]) y administración de transacciones atómicas junto con la DAL. Toda validación fallida queda registrada en la bitácora con nivel Warning.
class	SeguridadLogic	Lógica de seguridad (REQ-ARQ-006): autenticación centralizada, autorización por permisos y política anti fuerza bruta (5 intentos fallidos consecutivos → bloqueo de 15 minutos). Auditoría completa en bitácora de accesos exitosos y fallidos.

Espacio de nombres Services.BLL.Backups

Tipo	Nombre	Descripción
class	BackupLogic	Lógica del módulo de respaldo (REQ-ARQ-005): resuelve carpeta y nombre de archivo, verifica espacio en disco, ejecuta FULL/DIFFERENTIAL con CHECKSUM (el completo COPY_ONLY), verifica cada archivo con RESTORE VERIFYONLY, lleva el registro en la tabla Respaldos, restaura con copia previa de seguridad y administra el respaldo automático diario (solo con datos sanos, conservando los últimos 10). Cada operación queda registrada en la bitácora.

Espacio de nombres Services.BLL.ExceptionManagement

Tipo	Nombre	Descripción
class	ExceptionManagerLogic	Núcleo de gestión de excepciones (REQ-ARQ-004): registra los errores en la bitácora, muestra al usuario un mensaje amigable y ofrece reintentos ante fallos transitorios. Nunca debe lanzar excepciones secundarias.

Espacio de nombres Services.BLL.Infrastructure

Tipo	Nombre	Descripción
class	CryptographyLogic	Núcleo criptográfico del módulo Services (REQ-ARQ-002). Cifrado simétrico AES-256 (CBC + IV aleatorio por operación, PKCS7) y hashing de contraseñas con PBKDF2-SHA256 y sal aleatoria por usuario ("números semilla"). Formato del texto cifrado: Base64([IV 16 bytes] + [ciphertext]). Formato del hash: "salBase64.iteraciones.hashBase64".
class	CryptoKeyProvider	Provee la clave maestra de cifrado de la aplicación (REQ-ARQ-002). Cadena de resolución: 1) Variable de entorno ACO_ENC_KEY (desarrollo / demo; se hashea para obtener 32 bytes). 2) Archivo local protegido con DPAPI (Windows Data Protection API, usuario actual). 3) Si no existe: se genera una clave aleatoria de 256 bits y se persiste protegida con DPAPI. La clave nunca vive en el repositorio ni en texto plano en disco.

Espacio de nombres Services.BLL.Localization

Tipo	Nombre	Descripción
class	LocalizationLogic	Núcleo de internacionalización (T05). Los textos de los idiomas que se entregan (es, en, zh-CN) están en archivos .resx: Textos.resx es el idioma base y cada idioma tiene el suyo (Textos.en.resx, Textos.zh-CN.resx), que se compilan en ensamblados satélite, una carpeta por cultura junto al programa. La base de datos (tablas Idiomas y Textos) agrega idiomas nuevos y permite corregir leyendas sin recompilar: sus textos pisan a los del .resx. El cambio de idioma es dinámico, en caliente. Aplica el patrón observer: al cambiar el idioma (o editar las leyendas del idioma activo desde la administración) notifica a los suscriptores mediante el evento IdiomaCambiado.

Espacio de nombres Services.DAL.Factory

Tipo	Nombre	Descripción
class	RepositorioFactory	Fábrica de repositorios con reflexión. Lee del archivo de configuración el nombre completo de la clase que implementa cada interfaz (clave «Repositorio.NombreDeLaInterfaz») y crea la instancia. Valida que la clase exista, sea concreta, implemente la interfaz y tenga el constructor pedido; si algo falla lanza con un mensaje que dice qué corregir. Las clases de acceso a datos son internal: esta fábrica y las interfaces son la única puerta.

Espacio de nombres Services.DAL.Implementations

Tipo	Nombre	Descripción
class	FileLoggerRepository	Persistencia de respaldo de bitácora en archivo de texto con rotación automática (10 MB). Se usa cuando la escritura en base de datos falla (REQ-ARQ-003).
class	SqlBackupRepository	Operaciones de respaldo y restauración sobre SQL Server (REQ-ARQ-005). Usa la conexión "BackupString" (credenciales con permisos de administrador, exigidas por el ERS) y encapsula los comandos SQL nativos de BACKUP/RESTORE.
class	SqlLocalizacionRepository	Repositorio ADO.NET de idiomas y textos (T05): el contenido de localización vive en la base (tablas Idiomas y Textos) y se administra con comandos parametrizados.
class	SqlLoggerRepository	Persiste entradas de bitácora en la tabla Logs de la base Services (ADO.NET, REQ-ARQ-003). En caso de error técnico lanza . Expone también la consulta de las últimas entradas (pantalla de administración).
class	SqlPermisoRepository	Repositorio ADO.NET del sistema de permisos (T04): catálogo en árbol (tabla Permisos) y asignaciones por perfil (tabla PerfilesPermisos).
class	SqlRespaldoRegistroRepository	Registro de respaldos en la tabla Respaldos, con ADO.NET parametrizado vía .
class	SqlUsuarioRepository	Persistencia de usuarios y permisos en la base Services (ADO.NET, REQ-ARQ-006).

Espacio de nombres Services.DAL.Interfaces

Tipo	Nombre	Descripción
interface	IBackupRepository	Contrato de operaciones de respaldo/restauración sobre SQL Server (REQ-ARQ-005). La implementación encapsula los comandos SQL nativos (BACKUP/RESTORE...) y se ejecuta con credenciales de administrador (cadena de conexión "BackupString").
interface	ILocalizacionRepository	Contrato de acceso a datos de localización (T05): idiomas y textos almacenados en la base. Los consumidores (BLL) trabajan contra esta interfaz para mantener el modelo desacoplado del mecanismo de persistencia.
interface	ILoggerConsultaRepository	Repositorio de bitácora que además permite consultar las últimas entradas (tabla Logs).
interface	ILoggerRepository	Contrato de persistencia de bitácora (tabla Logs o archivo de respaldo).
interface	IPermisoRepository	Contrato de acceso a datos del sistema de permisos (T04): catálogo en árbol y asignaciones por perfil, almacenados en la base.
interface	IRespaldoRegistroRepository	Contrato del registro de respaldos (tabla Respaldos de OpenRIN_Services).
interface	IUsuarioRepository	Contrato de persistencia de usuarios y permisos (REQ-ARQ-006).

Espacio de nombres Services.DAL.Tools

Tipo	Nombre	Descripción
class	SqlHelper	Helper ADO.NET del módulo Services: ejecuta comandos parametrizados contra SQL Server, normaliza valores nulos y envuelve errores técnicos en para no filtrar detalles de conexión hacia las capas superiores (REQ-ARQ-004). Permite elegir la conexión configurada (por defecto "ServicesDB"; los respaldos usan "BackupString") y se ENLISTA automáticamente en la transacción activa (TransaccionDAL) cuando la conexión coincide (REQ-ARQ-007).
class	TransaccionDAL	Contexto de transacción de la capa de datos (REQ-ARQ-007): la BLL inicia una transacción y todos los métodos de escritura de la DAL que usen la MISMA conexión configurada se enlistan automáticamente (SqlHelper la detecta y reutiliza conexión + transacción). Alcance: una transacción activa por proceso (suficiente para la aplicación de escritorio).

Espacio de nombres Services.DomainModel

Tipo	Nombre	Descripción
class	Idioma	Idioma soportado por la aplicación (T05): vive en la base de datos y puede incorporarse o desactivarse desde la administración del sistema en cualquier momento.
class	LogEntry	Entrada de bitácora: un evento registrado por cualquier capa del sistema (REQ-ARQ-003).
enum	LogLevel	Niveles de severidad de bitácora (REQ-ARQ-003), de menor a mayor criticidad.
class	Permiso	Componente del patrón Composite (T04): un permiso puede ser atómico (representa una funcionalidad) o compuesto (agrupa a un conjunto de permisos). El código único identifica al permiso dentro del sistema (por ejemplo, "PA002" o "GE010").
class	PermisoAtomico	Permiso atómico (hoja del composite): representa una funcionalidad concreta.
class	PermisoCompuesto	Permiso compuesto (nodo del composite): agrupa un conjunto de permisos.
record	PermisoFila	Fila plana del catálogo de permisos (para construir el árbol en memoria).
class	RespaldoRegistro	Fila de la tabla Respaldos: un archivo .bak generado, restaurado o importado.
class	TextoLocalizacion	Texto de localización (T05): una leyenda concreta en un idioma concreto, almacenada en la base de datos. La clave identifica la leyenda en el código y el valor es lo que ve el usuario.
enum	TipoBackup	Tipo de respaldo soportado por el módulo de backups (REQ-ARQ-005).
class	Usuario	Usuario del sistema (tabla Usuarios de la base Services). Incluye el estado de acceso utilizado por la política anti fuerza bruta (REQ-ARQ-006) y el idioma preferido (REQ-ARQ-001).
class	UsuarioAutenticado	Sesión iniciada (REQ-ARQ-006): datos mínimos del usuario disponibles para toda la aplicación una vez autenticado.
record	UsuarioListado	Vista de usuario para la administración del sistema (pantalla Sistema): expone solo datos no sensibles (sin hash de contraseña ni pregunta de seguridad).

Espacio de nombres Services.DomainModel.Exceptions

Tipo	Nombre	Descripción
class	AccesoADatosException	Error de acceso a datos (REQ-ARQ-004): envuelve errores técnicos (por ejemplo, SqlException) y expone un mensaje amigable, ocultando los detalles de conexión hacia las capas superiores.
class	ConfiguracionInvalidaException	Falta o es incorrecta una clave de configuración (cadena de conexión, clase de repositorio...).
class	CredencialesInvalidasException	Credenciales inválidas (REQ-ARQ-006): usuario inexistente, contraseña incorrecta o usuario deshabilitado. Mensaje amigable, sin revelar detalles internos.
class	DatoInvalidoException	Un dato recibido no cumple una regla (vacío, fuera de rango, formato incorrecto).
class	IntegridadVulneradaException	Los dígitos verificadores indican que la base fue alterada fuera del sistema.
class	RegistroDuplicadoException	Ya existe un registro con el mismo valor único (usuario, idioma, DNI...).
class	RegistroNoEncontradoException	El registro pedido no existe.
class	ReglaNegocioException	Base de las excepciones controladas del sistema: su mensaje está pensado para el usuario, por eso la pantalla lo muestra tal cual en lugar de «Error inesperado».
class	RespaldoException	Falló un respaldo, una verificación o una restauración.
class	UsuarioBloqueadoException	Cuenta bloqueada temporalmente por la política anti fuerza bruta (REQ-ARQ-006): 5 intentos fallidos consecutivos → bloqueo de 15 minutos.

Espacio de nombres Services.DomainModel.Validacion

Tipo	Nombre	Descripción
class	RangoAttribute	Marca una propiedad numérica con un rango válido [Mínimo, Máximo] (REQ-ARQ-007). Ejemplo de uso: [Rango(0.5, 10.0)].
class	RequeridoAttribute	Marca una propiedad como obligatoria (REQ-ARQ-007): el valor no puede ser nulo ni, en el caso de cadenas, vacío o compuesto solo por espacios.
class	UnicoAttribute	Marca una propiedad cuyo valor debe ser único (REQ-ARQ-007). La verificación real de unicidad (contra la base de datos) la aporta el invocante mediante un delegado al llamar a ValidarEntidad.

Espacio de nombres Services.Facade

Tipo	Nombre	Descripción
class	BackupService	Fachada pública del módulo de respaldo (REQ-ARQ-005): crear, restaurar y verificar copias de seguridad de las bases de la plataforma, el registro de respaldos y el respaldo automático diario.
class	BitacoraService	Fachada pública de la bitácora (REQ-ARQ-003): registro de eventos de todas las capas y consulta de las últimas entradas para la pantalla de administración del sistema.

Tipo	Nombre	Descripción
class	ConsistenciaService	Fachada pública de consistencia (REQ-ARQ-007): validación de entidades por atributos ([Requerido], [Unico], [Rango]) y transacciones atómicas para las operaciones de escritura.
class	CryptographyService	Fachada pública del módulo de criptografía (REQ-ARQ-002). Punto de entrada único para cifrado, hashing y verificación de contraseñas.
class	ExceptionManager	Fachada pública de gestión de excepciones (REQ-ARQ-004). Único punto de entrada para manejar errores no controlados y reintentos.
class	LocalizationService	Fachada pública de internacionalización (T05): punto de entrada único para obtener textos traducidos, cambiar el idioma en caliente (notificando a los observadores suscriptos) y administrar idiomas y leyendas desde el sistema. Los textos de es, en y zh-CN están en archivos .resx (ensamblados satélite) y la base de datos agrega idiomas y corrige leyendas.
class	SeguridadService	Fachada pública del módulo de seguridad (REQ-ARQ-006): autenticación centralizada, verificación de permisos, registro de usuarios y listado para administración. Consumida por el formulario de login del Negocio, el portal de pacientes y la pantalla de administración del sistema.
class	SesionActual	Sesión activa del sistema (patrón Singleton, requisito T02): instancia única que conserva el usuario autenticado y expone utilidades de autorización. La creación es perezosa y sincronizada con lock (segura ante accesos concurrentes: base preparada para la incorporación de procesamiento multi-hilo).